University of Westminster
School of Computer Science & Engineering

6SENG005W Formal Methods (2026/27) Coursework

Module Leader	Subhaka Bhavanishankar (subhaka.b@iit.ac.lk)
Unit	**Coursework**
Weighting:	50%
Qualifying mark	30%
Description	To develop a B specification of a chess tournament using the B tools.
Learning Outcomes Covered in this Assignment:	The coursework assesses learning outcomes: LO1, LO2, LO3 & LO4.
Handed Out:	October 2026
Due Date	**13:00, Tuesday, 17th November, 2026**
Expected deliverables	Chess tournament B Specification: B machine(s) (mch file)
Method of Submission:	Online via Blackboard
Type of Feedback and Due Date:	Written feedback and marks available within 15 working days of submission. **All marks will remain provisional until formally agreed by an Assessment Board.**

Assessment regulations

Refer to section 4 of the "How you study" guide for undergraduate students for a clarification of how you are assessed, penalties and late submissions, what constitutes plagiarism etc.

Penalty for Late Submission

If you submit your coursework late but within 24 hours or one working day of the specified deadline, 10 marks will be deducted from the final mark, as a penalty for late submission, except for work which obtains a mark in the range 40 – 49%, in which case the mark will be capped at the pass mark (40%). If you submit your coursework more than 24 hours or more than one working day after the specified deadline you will be given a mark of zero for the work in question unless a claim of Mitigating Circumstances has been submitted and accepted as valid.

It is recognised that on occasion, illness or a personal crisis can mean that you fail to submit a piece of work on time. In such cases you must inform the Campus Office in writing on a mitigating circumstances form, giving the reason for your late or non-submission. You must provide relevant documentary evidence with the form. This information will be reported to the relevant Assessment Board that will decide whether the mark of zero shall stand. For more detailed information regarding University Assessment Regulations, please refer to the following website:

http://www.westminster.ac.uk/study/current-students/resources/academic-regulations

Academic Integrity

Academic integrity means being honest, fair and responsible in your academic work. This includes completing your own assignments, exams and research, and clearly acknowledging any ideas, words or information you have used from other people or sources, for example through referencing.

If there is uncertainty regarding the authenticity of your work, a viva meeting may be held to verify the authenticity.

The meeting is a conversation between you and two academics.

A viva gives you the opportunity to:

- explain your ideas and the choices you made in your work;
- show that you understand the sources and information you used;
- show that the work was completed independently.

At the meeting, you may be asked to discuss or demonstrate parts of your work, explain how you researched and prepared it, and describe the sources or tools you used.

Before the meeting, you will have the opportunity to provide any documents or information you think are relevant. This could include draft versions of your work, research notes, plans, or other materials that show how you developed the work.

After the meeting one of the following decisions will be taken.

a. that the work will be referred for further investigation under Section 10 Academic Misconduct regulations

b. that there are no concerns regarding the submitted work, in which case the work will be marked as normal.

If you do not attend the meeting the work will be treated as a non-submission and a mark of zero will be recorded for the assessment component.

Further information: Academic misconduct | University of Westminster, London

Viva Examination Requirement:

The viva examination constitutes a critical component of this coursework assessment.
Marks will be allocated primarily based on the student's individual performance and ability to
demonstrate a clear understanding of their submitted work during the viva.
Failure to adequately justify or explain the submission, or inability to respond appropriately to viva questions, may result in a substantial reduction of marks or a fail grade for the module, irrespective
of the quality of the submitted report or implementation.

Coursework description

1. Introduction

This coursework requires you to develop a B specification of a chess tournament.

The tournament involves 12 players, listed in decreasing order of ability (i.e. Alice is the strongest player and Louis is the weakest):

> Alice, Bob, Christine, Dennis, Eloise, Frank, Gabrielle, Hugo, Ida, John, Kate, Louis.

It proceeds in three phases: **team selection**, **match setup**, and **game play**.

During the **team selection** phase, players are sorted into three teams of 4: **red**, **blue**, and **green**.

Once teams are complete, **matches are set up** between these teams. Finally, the matches are **played out** and scores computed. After this, we can start over from the team selection phase.

1.1 The Chess Tournament: Data Constraints and Requirements

For the purposes of this coursework, you need to create a B-machine representing the following data (making sure to choose appropriate types):

- The players as listed above:
  - Alice, Bob, Christine, Dennis, Eloise, Frank, Gabrielle, Hugo, Ida, John, Kate, Louis
- Their relative levels of ability (Alice being stronger than Bob etc)
- The team names (red, blue, green)
- The current phase (team selection, match setup, game play)
- The current members of each team
  - each having up to 4 of the players listed above
  - initially none
- The currently scheduled matches
  - each involving two players from different teams
  - up to 6 per player and up to 12 between each pair of teams
  - initially none
  - Note that some operations require you to keep track of the order in which the matches were scheduled.
- The scores for each player and each team, initially 0

1.2 The Chess Tournament: Operations

The machine should offer the following operations:

`report <-- setPlayers(team, members)`

> Tries to add the players in **`members`** to the team. Succeeds if the following conditions are satisfied:

- The current phase is **team selection**
- The team does not yet have any players
- There are exactly four players in **`members`**
- None of them are already in a team

If this is the **second team** that has been completed, the remaining players should be **automatically added** to the one team that is still empty, and the current phase becomes **match setup**.

The operation reports **teamsCompleted, success**, or an **error message** indicating what the error was.

`report <-- setupMatch(players)`

Tries to set up a match between the given pair of players and add it to the currently scheduled matches. Succeeds if the following conditions are satisfied:

- These two players do not yet have a match scheduled against each other
- The players belong to different teams
- Neither player already has 6 matches scheduled
- There are less than 12 matches currently scheduled between these players' teams

Then the operation either reports **success** or an **error message** indicating what the error was.

`report <-- undo`

Undoes the most recently scheduled match, if any. Succeeds if at least one match is currently scheduled.

The operation reports **success** or **failure** as appropriate.

`proceed`

Enters the **game play** phase. Succeeds if the following conditions are satisfied:

- Each player has 6 currently scheduled matches
- There are 12 currently scheduled matches between each pair of teams

The operation reports **success** or **failure** as appropriate.

`nextGame`

Plays the first of the still scheduled games, updating the winner's score and removing this game from the schedule.

- Succeeds if we are in the **game play** phase and there is still at least one scheduled game
- For simplicity we assume that the more skilled player always wins.

The operation reports **success** or **failure** as appropriate.

`restart`

Returns to the **team selection** phase, resetting all teams and scores. **Can only be used** if the following conditions are satisfied:

- We are in the game play phase
- There are no more scheduled matches

`playerScores(team)`

Reports the current scores for all players in the given team.

2.2 General Requirements

The B machine should use the appropriate features to define the data and operations described above.

The specification must be syntactically and type correct and work when simulated in ProB. That is, it must initialise correctly and all operations must work as specified above to model the tournament.

2.3 Blackboard Submission

Upload a single mch file directly on blackboard.

Marking Scheme Overview

The Coursework will be marked based on the following main component marking criteria, the full marking criteria details will be published on the module Blackboard site.

Criteria	Mark per component
Sets and constants	**15**
Variables, invariant and initialisation	**15**
setPlayers operation	**10**
setupMatch operation	**10**
undo operation	**10**
proceed operation	**10**
nextGame operation	**10**
restart operation	**10**
playerScores operation	**10**
Total	**100**